

PIMCI Normal Form $NF(A)$: Kirby Generators, Discriminant Pairing, and Deterministic Canonicalization

This document formalizes:

1. **Kirby generators** as explicit matrix actions on a symmetric integral matrix A .
2. The **stable Kirby invariant** $(G(A), \lambda_A)$ where $G(A) = \mathbb{Z}^n / A\mathbb{Z}^n$ and λ_A is the induced $\mathbb{Q}/\mathbb{Z}$ -valued pairing.
3. A **deterministic normal form map** NF such that for any Kirby word W ,

$$NF(W[A]) = NF(A).$$

4. An **implementable pseudocode pipeline**:

$$\text{SNF} \rightarrow \text{pairing matrix} \rightarrow p\text{-primary split} \rightarrow \text{canonical block reduction}.$$

5. An explicit **block library** for odd primes p and for $p = 2$ (finite abelian p -groups with nondegenerate symmetric pairings into $\mathbb{Q}/\mathbb{Z}$).

Scope: This is a *stable* Kirby normal form for the discriminant pairing. For unimodular A ($|\det A| = 1$), $G(A) = 0$ and the output is trivial; exotic-sphere sensitivity typically enters through an additional smooth channel κ (not expanded here).

1. Kirby generators as matrix actions

Let $A \in M_n(\mathbb{Z})$ be **symmetric**.

(G0) Relabeling

For a permutation matrix P ,

$$A \mapsto P^T A P.$$

(G1) Handle slide $i \leftarrow i \pm j$

Let E_{ij} have a single 1 at (i, j) . For $\sigma \in \{+1, -1\}$, define

$$U := I + \sigma E_{ij} \in GL_n(\mathbb{Z}).$$

Then

$$A \mapsto U^T A U.$$

Entrywise: - For $k \neq i$: $a'_{ik} = a_{ik} + \sigma a_{jk}$ and symmetry gives a'_{ki} similarly. - $a'_{ii} = a_{ii} + 2\sigma a_{ij} + a_{jj}$. - Other entries unchanged.

(G2) Stabilization

For $\varepsilon \in \{+1, -1\}$,

$$A \mapsto A \oplus [\varepsilon].$$

A **Kirby word** W is any composition of these operations.

2. The invariant: discriminant group and linking pairing

Assume $\det(A) \neq 0$ (nongenerate over $\mathbb{Q}$). Define

$$G(A) := \mathbb{Z}^n / A\mathbb{Z}^n.$$

This is finite of order $|\det(A)|$.

Define the induced **symmetric pairing**

$$\lambda_A : G(A) \times G(A) \rightarrow \mathbb{Q}/\mathbb{Z}, \quad \lambda_A(\bar{x}, \bar{y}) := x^T A^{-1} y \bmod 1.$$

Why Kirby moves preserve (G, λ)

- **Slides / relabeling:** $A' = U^T A U$ with unimodular U gives an induced isomorphism $\bar{x} \mapsto \overline{U^{-1}x}$ preserving λ .
- **Stabilization:** $[\pm 1]$ is unimodular so contributes a trivial discriminant summand.

Thus the isomorphism class of $(G(A), \lambda_A)$ is invariant under all Kirby words.

3. Definition of the normal form map NF

Definition (stable Kirby normal form). $NF(A)$ is a canonical encoding of the isomorphism class of the finite pairing $(G(A), \lambda_A)$.

Then for any Kirby word W :

$$NF(W[A]) = NF(A).$$

To implement NF , we produce a deterministic decomposition of $(G(A), \lambda_A)$ into canonical prime-power blocks.

4. Implementation pipeline overview

Input: symmetric integer matrix A with $\det(A) \neq 0$. $=$

Output: a structured object

$$NF(A) = \{ NF_p(A) : p \mid \det(A) \}$$

where each NF_p is a list of canonical p -primary pairing blocks.

Pipeline: 1. Compute **Smith normal form** (SNF) of A . 2. Compute an explicit **pairing matrix** for λ_A on a canonical SNF-derived basis. 3. Split into **p-primary components**. 4. For each p , perform **canonical block reduction** using the block library.

5. Step 1: Smith normal form (SNF)

Compute unimodular $S, V \in GL_n(\mathbb{Z})$ such that

$$SAV = D = \text{diag}(d_1, \dots, d_n), \quad d_i \mid d_{i+1}.$$

Then

$$G(A) \cong \bigoplus_{i: d_i \neq 1} \mathbb{Z}/d_i\mathbb{Z}.$$

The list (d_i) is canonical.

Let I be the index set of nontrivial invariants $\{i : d_i > 1\}$ and define

$$D_I := \text{diag}(d_i)_{i \in I}, \quad m := |I|.$$

We will represent elements of $G(A)$ by coordinate vectors $u \in \mathbb{Z}^m$ modulo D_I .

6. Step 2: pairing matrix in SNF coordinates

We need a matrix $P \in M_m(\mathbb{Q}/\mathbb{Z})$ such that for $u, v \in \mathbb{Z}^m$,

$$\lambda(u, v) = u^T P v \pmod{1}.$$

Constructing P

From $SAV = D$, note $A = S^{-1}DV^{-1}$, hence

$$A^{-1} = VD^{-1}S.$$

Therefore for lifts $x, y \in \mathbb{Z}^n$,

$$\lambda_A(\bar{x}, \bar{y}) = x^T V D^{-1} S y \bmod 1.$$

Restrict to the components in I : let π_I project to those indices. Define

$$P := \pi_I (V^T A^{-1} V) \pi_I^T \bmod 1.$$

Implementation-friendly version (avoids explicit A^{-1}):

$$P = \pi_I (V^T V D^{-1} S V) \pi_I^T \bmod 1.$$

In practice, compute over rationals with exact arithmetic and then reduce each entry to $[0, 1) \subset \mathbb{Q}$ representing its class mod 1.

Sanity: P is symmetric in $\mathbb{Q}/\mathbb{Z}$.

7. Step 3: p-primary split

For each prime $p \mid \det(A)$: - Decompose each d_i into $d_i = \prod_p p^{e_{i,p}}$. - The p -primary subgroup is

$$G_p \cong \bigoplus_{i \in I: e_{i,p} > 0} \mathbb{Z}/p^{e_{i,p}} \mathbb{Z}.$$

- Restrict the pairing matrix P to the coordinates with $e_{i,p} > 0$ and then project values to the p -primary torsion of $\mathbb{Q}/\mathbb{Z}$, i.e. reduce denominators to powers of p .

Concretely, for an entry $r \in \mathbb{Q}/\mathbb{Z}$ represented by a/b in lowest terms: - Keep only the p -power part: write $b = p^t b'$ with $\gcd(b', p) = 1$. - Replace a/b by $a(b')^{-1}/p^t \bmod 1$, where $(b')^{-1}$ is inverse mod p^t .

This yields a pairing λ_p on G_p .

8. Step 4: canonical block reduction

We now reduce (G_p, λ_p) to a canonical direct sum of standard blocks.

8.1 Odd prime block library (p odd)

Every nondegenerate symmetric pairing on a finite abelian p -group decomposes as an orthogonal direct sum of **cyclic blocks**:

• **Cyclic block** $C(p^k, \epsilon)$:

$$(\mathbb{Z}/p^k \mathbb{Z}, \langle \epsilon/p^k \rangle),$$

meaning in the generator basis g ,

$$\lambda(g, g) = \epsilon/p^k \bmod 1,$$

where $\epsilon \in (\mathbb{Z}/p^k\mathbb{Z})^\times$.

For odd p , ϵ is classified up to multiplication by a square; there are exactly **two** square-classes in $(\mathbb{Z}/p^k\mathbb{Z})^\times$: residue and nonresidue.

Canonical choice of ϵ : - Let η_p be the smallest positive integer that is a quadratic nonresidue mod p (and hence mod p^k by Hensel lifting for odd p). - Represent the two classes by $\epsilon = 1$ and $\epsilon = \eta_p$.

So the odd-prime library is

$$\mathcal{B}_p^{\text{odd}} = \{ C(p^k, 1), C(p^k, \eta_p) : k \geq 1 \}.$$

8.2 The $p = 2$ block library

For 2-primary groups, one needs both 1-dimensional and certain 2-dimensional indecomposables.

Use the following standard blocks.

(A) 1D cyclic blocks (diagonal types)

• $A(2^k, \epsilon)$:

$$(\mathbb{Z}/2^k\mathbb{Z}, \langle \epsilon/2^k \rangle)$$

where ϵ is odd.

Canonical representatives for ϵ modulo squares in $(\mathbb{Z}/2^k\mathbb{Z})^\times$ can be taken as: - for $k = 1$: only $\epsilon \equiv 1$ (since $\mathbb{Z}/2^\infty$ is trivial) - for $k = 2$: $\epsilon \in \{1, 3\}$ - for $k \geq 3$: $\epsilon \in \{1, 3, 5, 7\}$ (these represent the four square-classes of odd units mod 2^k).

So include

$$\mathcal{A}_{2,k} = \begin{cases} \{A(2, 1)\} & k = 1 \\ \{A(4, 1), A(4, 3)\} & k = 2 \\ \{A(2^k, 1), A(2^k, 3), A(2^k, 5), A(2^k, 7)\} & k \geq 3. \end{cases}$$

(B) 2D “hyperbolic” blocks

• $H(2^k)$: group $(\mathbb{Z}/2^k\mathbb{Z})^2$ with Gram matrix

$$\frac{1}{2^k} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad (\text{entries in } \mathbb{Q}/\mathbb{Z}).$$

(C) 2D “twisted hyperbolic” blocks

• $T(2^k)$: group $(\mathbb{Z}/2^k\mathbb{Z})^2$ with Gram matrix

$$\frac{1}{2^k} \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

Thus the 2-adic library is

$$\mathcal{B}_2 = \bigcup_{k \geq 1} (\mathcal{A}_{2,k} \cup \{H(2^k), T(2^k)\}).$$

Note: This library is standard for nondegenerate symmetric bilinear pairings on 2-groups; the diagonal-only family is not sufficient in general, hence H and T .

9. Deterministic canonical block reduction algorithm

We now give implementable pseudocode that takes: - $\{e_i\}$ generators with orders p^{k_i} , - a pairing matrix $P_p \in M_m(\mathbb{Q}/\mathbb{Z})$ whose entries have denominators powers of p ,

and returns a canonical multiset of blocks from $\mathcal{B}_p$.

9.1 Helper operations

- `order(i)`: returns p^{k_i} .
- `val_p(x)`: p-adic valuation of a rational number represented as a/p^t (so `val_p(a/p^t) = -t` if $p \nmid a$).
- `reduce_mod_1(q)`: reduce rational to $[0, 1)$.
- Basis change by unimodular matrix over $\mathbb{Z}$ respecting the abelian invariants: use elementary column/row operations corresponding to changing generators.

9.2 Odd p : diagonalization into cyclic blocks

For odd p , we can deterministically diagonalize.

Pseudocode (odd prime reduction):

```
function CanonicalBlocksOddP(p, orders[k1..km], P):
    blocks = []
    # Work from largest exponent down for determinism
    while m > 0:
        # Choose pivot generator i deterministically:
        # 1) max order, 2) smallest index among ties
        i = argmax_i(orders[i])
        k = log_p(orders[i])

        # Find j such that pairing with i has minimal p-valuation (strongest
        # coupling)
        # Prefer diagonal if nonzero.
```

```

    if P[i,i] != 0:
        j = i
    else:
        j = argmin_j(val_p(P[i,j])) over j with P[i,j] != 0

    # If j != i, change basis to make diagonal pivot nonzero (odd p allows
this)
    if j != i:
        # Replace generator i by i + t*j to make P[i,i] nonzero
        # Choose smallest t in {0..p-1} that works.
        for t in 0..p-1:
            if P[i,i] + 2*t*P[i,j] + t*t*P[j,j] != 0:
                apply basis change: e_i <- e_i + t*e_j
                update P accordingly
                break

    # Now diagonal entry P[i,i] is nonzero: it defines a cyclic block
    # Normalize to epsilon/p^k
    q = P[i,i] # element of Q/Z with denom p^k'
    # Reduce q to form a/p^k (k determined by order), ensure gcd(a,p)=1
    a, kk = normalize_to_unit_over_p(q) # q = a/p^kk with p|a
    # For a generator of order p^k, enforce kk == k by rescaling generator
if needed
    # (Implement by splitting off smaller-order components first.)

    # Determine square-class of a mod p:
    if is_quadratic_residue_mod_p(a, p):
        eps = 1
    else:
        eps = eta_p(p) # smallest nonresidue mod p

    blocks.append( C(p^k, eps) )

    # Orthogonalize: kill row/col i against all others
    for r != i:
        # choose u to eliminate P[r,i] using diagonal pivot P[i,i]
        u = P[r,i] / P[i,i] (computed in Z/p^k)
        apply basis change: e_r <- e_r - u*e_i
        update P

    # Remove i from the system
    delete row/col i from P
    delete orders[i]
    m = m - 1

# Sort blocks canonically by (k, eps) then return
return sort(blocks)

```

9.3 $p = 2$: reduction with 1D and 2D blocks

For 2, diagonalization may fail; we extract either a 1D diagonal block, or a 2D H or T block.

Deterministic extraction strategy: 1. Work by **largest order** first. 2. Prefer extracting a 1D block when you can find an element with $\lambda(x, x) \neq 0$. 3. Otherwise extract a 2D block from a pair with $\lambda(x, y)$ having maximal 2-adic denominator.

Pseudocode (2-adic reduction):

```
function CanonicalBlocks2(orders[2^k1..2^km], P):
    blocks = []
    while m > 0:
        i = argmax_i(orders[i])
        k = log2(orders[i])

        if P[i,i] != 0:
            # 1D extraction
            q = P[i,i] # in Q/Z with denom 2^t
            eps = canonical_odd_unit_class(q, k) # maps to {1} or {1,3} or {1,3,5,7}
            blocks.append( A(2^k, eps) )

            # Orthogonalize against others using pivot
            for r != i:
                u = solve_for_u_to_kill(P[r,i], P[i,i], 2^k)
                e_r <- e_r - u*e_i
                update P

            delete i
            continue

        # If diagonal is zero, try to form 2D block with some j
        # Choose j that maximizes denominator (i.e., minimizes valuation) of
        P[i,j]
        j = argmin_j(val2(P[i,j])) over j with P[i,j] != 0

        # Now classify the 2D form on span{i,j}
        # We have Gram matrix [[0, b],[b, c]] (after possible basis tweaks),
        with b != 0.
        # Make b = 1/2^k via scaling within allowed generator changes.
        normalize_pair(i, j) # deterministic basis changes to standardize
        cross-term

        # Decide whether this is H(2^k) or T(2^k)
        if can_make_diagonals_zero():
```



```

        blocks.append( H(2^k) )
    else:
        blocks.append( T(2^k) )

    # Orthogonalize remaining generators against span{i,j}
    for r not in {i,j}:
        eliminate_pairings_with_span(r, i, j)

    delete i and j

return sort(blocks)

```

Notes for implementers (2-adic details): - `canonical_odd_unit_class(q,k)`: - write $q = a/2^t$ with odd a . Reduce to modulus 2^k and map a into the canonical representative set: - $k = 1$: $\{1\}$ - $k = 2$: $\{1,3\}$ - $k \geq 3$: $\{1,3,5,7\}$ - `normalize_pair(i,j)` and `can_make_diagonals_zero()` are standard 2-adic basis adjustments on the 2D span; implement deterministically by trying a fixed ordered list of substitutions (e.g., $e_i \leftarrow e_i + te_j$ for $t \in \{0,1\}$, then $e_j \leftarrow e_j + te_i$, etc.) and selecting the first that matches the target pattern.

10. Full pseudocode for $NF(A)$

```

function NF(A):
    assert A is symmetric integer matrix
    assert det(A) != 0

    # Step 1: SNF
    S, D, V = SmithNormalForm(A) # S*A*V = D diag(d1..dn)
    I = [i : D[i,i] > 1]
    d = [D[i,i] for i in I]

    if len(I) == 0:
        return {} # unimodular: trivial discriminant

    # Step 2: pairing matrix in SNF coords
    # Use  $A^{-1} = V * D^{-1} * S$ 
    # Compute  $B = V^T * A^{-1} * V = V^T * V * D^{-1} * S * V$ 
    B = transpose(V) * V * inverse_diag(D) * S * V # exact rationals
    P = restrict(B, I, I) # m x m rational matrix
    P = entrywise_reduce_mod_1(P)

    # Step 3: p-primary split
    primes = prime_factors(abs(det(A)))
    NF_out = {}

    for p in sorted(primes):

```

```

# indices in I whose invariant factor has p-power
Ip = [t in 1..m such that v_p(d[t]) > 0]
if Ip empty:
    continue

orders = [p^{v_p(d[t])} for t in Ip]
Pp = restrict(P, Ip, Ip)
Pp = project_pairing_to_p_primary(Pp, p)

# Step 4: canonical block reduction
if p != 2:
    blocks = CanonicalBlocksOddP(p, orders, Pp)
else:
    blocks = CanonicalBlocks2(orders, Pp)

NF_out[p] = blocks

return NF_out

```

Output format suggestion: for each prime p , return a list like - odd p : $[(k, \text{eps_tag}, \text{multiplicity}), \dots]$ where $\text{eps_tag} \in \{\text{res}, \text{nonres}\}$ - $p = 2$: $[(\text{type}, k, \text{params}, \text{multiplicity}), \dots]$ where $\text{type} \in \{A, H, T\}$ and params is the canonical ϵ if type A.

11. Determinism rules (to guarantee identical output)

Whenever there is a choice: 1. Prefer **largest-order** generators first. 2. Break ties by **smallest index**. 3. For basis changes, try coefficients in a fixed ordered set (odd p : $t = 0..p - 1$; $p = 2$: $t \in \{0, 1\}$) and take the first that achieves the target. 4. Sort blocks canonically at the end by a fixed key: - odd p : sort by $(k, \text{eps_tag})$ - $p = 2$: sort by $(\text{type_order}, k, \text{params})$ with $A < H < T$.

These rules ensure NF is a deterministic function, not just an isomorphism class.

12. Invariance theorem (the guarantee)

Theorem. Let $\sim$ be the equivalence relation generated by Kirby generators (G0-G2). If $A \sim A'$, then $NF(A) = NF(A')$.

Reason. Each generator preserves the isomorphism class of $(G(A), \lambda_A)$. The pipeline computes a canonical representative of that isomorphism class, hence is invariant under any Kirby word.

13. Practical caution for exotic spheres

If $|\det A| = 1$, the discriminant pairing is trivial and $NF(A)$ will be empty.

For exotic-sphere detection you typically extend the carrier with a **smooth refinement channel** κ (e.g., Pontryagin/characteristic correction). Then define

$$NF_{\text{smooth}}(A, \kappa) := (NF(A), NF(\kappa \text{ in matching prime tiers})).$$

This document provides the stable Kirby normal form part $(NF(A))$.